Multi-Agent LLM Code Optimization with Tree Search and Shared Memory

Anonymous

Abstract

We present SWARMOPT, a framework for automated code optimization that combines multiple LLM agents with tree search and a shared memory board. Given a target program and a numeric evaluation function, SWARMOPT iteratively generates code modifications through diverse agents—each with a distinct optimization strategy and sampling temperature—that share findings via a persistent board. When optimization stalls, a backtracking mechanism restores promising earlier states and redirects search to unexplored regions of the solution space. Across four benchmarks (game AI, bioinformatics, job scheduling, and TSP), the full system achieves up to $+282\%$ improvement over baselines. Ablations over 100 runs show that multi-agent diversity contributes up to $2.8\times$ larger gains than single-agent search, and tree-based backtracking adds $+27\%$ relative improvement on complex tasks. Parallel execution provides $2.6\times$ wall-clock speedup with no quality loss.

1 Introduction

Large language models (LLMs) can generate and modify code at human level [7], yet their application to *iterative code optimization*—repeatedly improving a program against a quantitative objective—remains underexplored. Prior work has shown that LLMs can discover novel algorithms through evolutionary search [10] or improve code via self-refinement [8], but these approaches typically use a single model in a linear chain of revisions, risking convergence to local optima.

We identify three limitations of single-agent iterative optimization: (1) **Strategy collapse**—a single agent tends to repeat similar modifications, exhausting one region of the solution space; (2) **No recovery from dead ends**—once a promising direction stalls, there is no mechanism to backtrack; (3) **No knowledge transfer**—failed attempts are discarded rather than informing future exploration.

SWARMOPT addresses these limitations through three mechanisms:

- **Agent diversity.** Multiple agents with distinct strategies and sampling temperatures reduce strategy collapse.
- **Shared memory board.** A persistent board records all experiments, enabling cross-agent knowledge transfer and failure avoidance.
- **Tree search with backtracking.** The optimization trajectory is maintained as a tree of program states. When progress stalls, the system backtracks to a high-scoring unexplored node (Figure 1).

We evaluate on four diverse benchmarks and conduct systematic ablations (100 runs total) isolating each component's contribution. On complex tasks, the full system significantly outperforms both single-agent and no-backtracking variants.

2 Related Work

LLM-based code optimization.. FunSearch [10] evolves mathematical programs via LLMs, maintaining a population of solutions ranked by fitness. AlphaCode [7] generates massive candidate pools and filters via execution. EvoPrompting [2] combines evolutionary search with LLM mutation for neural architecture search. These approaches use flat populations; SWARMOPT maintains a *structured search tree* enabling targeted backtracking to promising states.

Multi-agent LLM systems.. ChatDev [9] and MetaGPT [5] assign specialized roles in software development pipelines. Multi-agent debate [4] improves reasoning through discussion. AgentCoder [6] separates generation from testing. These systems optimize for *correctness* through role specialization;

SWARMOPT optimizes a *numeric objective* with strategy-diverse agents sharing results through a common board.

Search and self-refinement.. Tree of Thoughts [13] applies tree search to LLM reasoning at the thought-step level. Reflexion [11] uses verbal self-reflection; Self-Refine [8] iterates generation and critique. Large Language Monkeys [1] scales inference via repeated sampling. Our tree search operates at the *program state* level, with backtracking restoring actual file contents rather than reasoning steps.

3 Method

3.1 Problem Formulation

Given target source files $\mathbf{f} = \{f_1, \ldots, f_m\}$, an evaluation function $\texttt{eval}(\mathbf{f}) \to \mathbb{R}$, and a direction $d \in \{\max, \min\}$, find $\mathbf{f}^*$ optimizing $\texttt{eval}(\mathbf{f}^*)$. The evaluation function is a black-box executable returning a numeric score.

3.2 Multi-Agent Architecture

The system maintains N agents, each defined by a strategy description s_i and sampling temperature τ_i. The default pool uses three agents: **Explorer** (τ=0.9)—bold, creative approaches; **Optimizer** (τ=0.3)—focused, incremental improvements; **Synthesizer** (τ=0.6)—combines insights from the board.

In **sequential mode**, agents run in order. Each sees the latest file state (including changes kept by earlier agents in the same round), the board, and its own memory. Kept changes accumulate; reverted changes inform the board. In **parallel mode**, agents receive the same snapshot and run concurrently in isolated copies; the best improvement is committed. This trades the sequential cascade for wall-clock speedup.

3.3 Shared Memory Board

The board persistently stores every experiment result: agent, round, score, delta, kept/reverted status, and a change summary. It provides two views injected into prompts: (1) *success history*—all kept changes with scores; (2) *failure blocklist*—reverted changes with an explicit "do not retry" instruction. This enables cross-agent knowledge transfer: an Explorer's algorithmic discovery informs the Optimizer's refinement.

3.4 Tree Search with Backtracking

The optimization trajectory forms a tree T where each node v stores a file state $\mathbf{f}_v$, score $\texttt{eval}(\mathbf{f}_v)$, creating agent, and visit metadata. Kept modifications add child nodes; the active pointer advances to the new child.

Backtrack trigger.. A counter tracks consecutive stale rounds (no improvement). When stale rounds $\geq k$, the system backtracks.

Node selection.. The target is selected by scoring all nodes not on the current root-to-active path:

$$\text{priority}(v) = \texttt{score}(v) \cdot \frac{1}{1 + \text{visits}(v)} \cdot \begin{cases} 0.3 & \text{if abandoned} \\ 1.0 & \text{otherwise} \end{cases} \tag{1}$$

This balances exploitation (high scores) with exploration (fewer visits) while penalizing abandoned branches—analogous to UCB in MCTS [3], but over program states rather than game actions.

State restoration.. On backtrack, the current branch is marked abandoned, files are restored to the target node, and agents receive context describing abandoned branches to encourage divergence. A global best is tracked across all branches and restored at termination.

4 Experiments

4.1 Setup

We evaluate on four tasks of varying complexity:

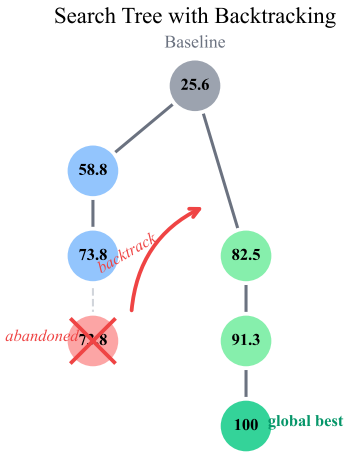


Figure 1: Search tree with backtracking. Nodes store program states and scores. When the left branch stalls (abandoned), the system backtracks to an earlier node and explores a new direction, ultimately finding the global best.

Table 1: Component ablation. Best mean per task in **bold**. Game-AI and Scheduler: n=7; Bio-Opt and TSP: n=3.

Task	Config	Baseline	Mean ± Std	Δ%
Game-AI ↑	Full	25.6	**87.5 ± 16.1**	+241%
	No-Backtrack	25.6	80.5 ± 9.4	+214%
	Single-Agent	25.6	80.4 ± 10.1	+214%
Bio-Opt ↑	Full	39.6	**49.9 ± 6.2**	+26%
	No-Backtrack	39.6	45.3 ± 6.0	+14%
	Single-Agent	39.6	43.2 ± 3.2	+9%
Scheduler ↓	Full	213,837	87,697 ± 25,737	−59%
	No-Backtrack	213,837	**84,963 ± 24,849**	−60%
	Single-Agent	213,837	100,225 ± 27,762	−53%
TSP ↓	Full	592.7	**494.8 ± 2.5**	−17%
	No-Backtrack	592.7	497.6 ± 4.5	−16%
	Single-Agent	592.7	495.6 ± 0.5	−16%

- **Game-AI**: Othello agent vs. 4 opponents (2 files). Win rate (%), baseline 25.6%.
- **Bio-Opt**: DNA motif finding. F1-like score, baseline 39.6.
- **Scheduler**: Job shop, 50 jobs × 10 machines (3 files). Weighted tardiness (min), baseline 213,837.
- **TSP**: 40-city traveling salesman (3 files). Tour distance (min), baseline 592.7.

All experiments use Kimi K2.5 as the LLM, 6 rounds, 3 default agents, backtrack threshold k=3, max backtracks = 5. Each config runs n=3–7 times. We report mean ± std and improvement over baseline.

4.2 Component Ablation

We compare **Full** (3 agents + backtracking), **No-Backtrack** (3 agents, k=0), and **Single-Agent** (Explorer only, with backtracking).

Results are shown in Table 1 and Figure 2.

Multi-agent diversity.. Across all tasks, multi-agent configs outperform Single-Agent. The effect is strongest on Bio-Opt: Full achieves +26% vs. Single-Agent's +9%—a 2.8× larger gain. On Scheduler, multi-agent configs reduce tardiness by ∼60% vs. 53%. On Game-AI, Full reaches 100%

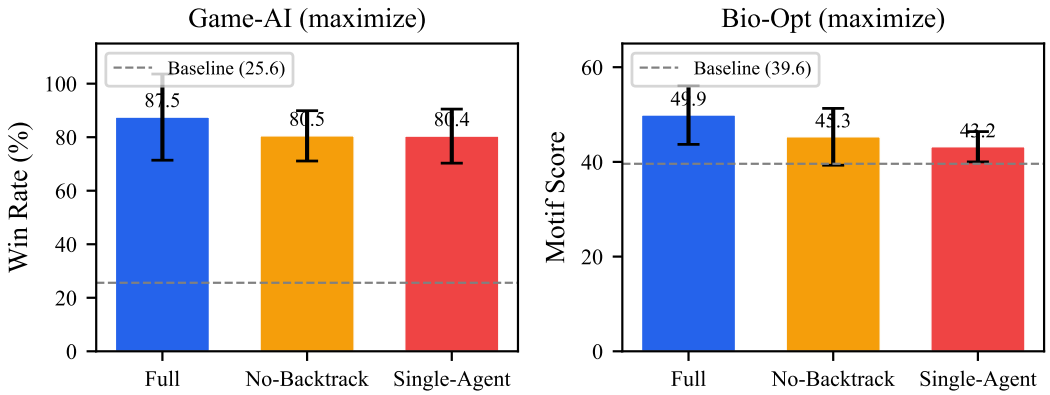
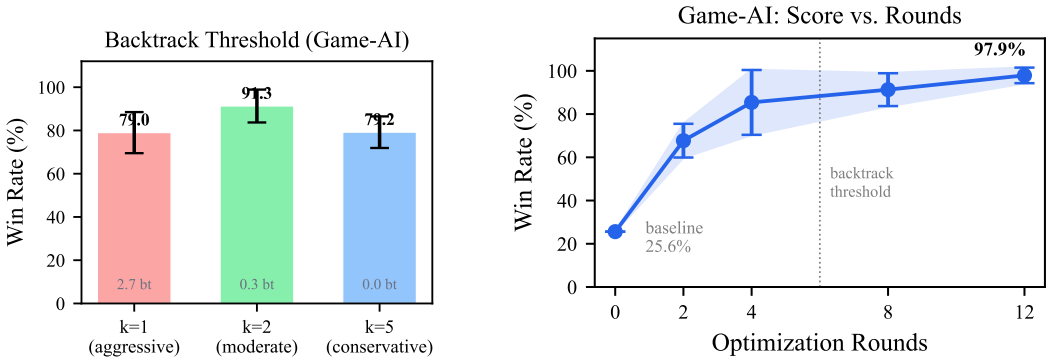


Figure 2: Component ablation on Game-AI and Bio-Opt. Full system (blue) consistently outperforms No-Backtrack (orange) and Single-Agent (red). Dashed lines show baselines. Error bars: ± 1 std.



(a) Backtrack threshold k (n=3, 6 rounds). Moderate k=2 is optimal; aggressive (k=1) wastes rounds on frequent restarts.

(b) Score vs. optimization rounds (n=3, full config). Main gains at 2–4 rounds; backtracking extends progress at 8+.

Figure 3: Sensitivity analysis on Game-AI.

win rate in 4/7 runs vs. 1/7 for both alternatives.

Backtracking.. On Game-AI, Full outperforms No-Backtrack by $+27\%$ relative (87.5 vs. 80.5; Cohen's d=0.53). On Bio-Opt, the gap is $+12\%$ relative. On Scheduler and TSP, backtracking adds minimal benefit—these tasks either have smooth landscapes (TSP) or high variance that masks the signal (Scheduler).

4.3 Backtrack Threshold and Round Scaling

Figure 3a shows that backtrack threshold k=2 is optimal on Game-AI (91.3 vs. $\sim$79 for $k \in \{1, 5\}$). Setting k=1 triggers 2.7 backtracks per run, wasting rounds on state restoration. Setting k=5 never triggers within 6 rounds, effectively disabling the mechanism.

Figure 3b shows diminishing returns with more rounds. The largest marginal gain is from 2 to 4 rounds ($+18$ points). Backtracking activates at round 8+ (threshold k=3), providing continued gains that push the system to 97.9% at 12 rounds (2/3 runs at 100%).

4.4 Parallel Execution

Sequential and parallel modes achieve identical quality on Game-AI (81.7 ± 6.9 vs. 81.5 ± 6.3) while parallel provides $2.6\times$ wall-clock speedup (429s vs. 1122s, n=3). Parallel mode selects the best among concurrent proposals, compensating for loss of the sequential improvement cascade.

5 Discussion and Limitations

Task-dependent contributions.. Component contributions vary by task. Multi-agent diversity helps consistently across all benchmarks. Backtracking helps most on complex tasks with rugged landscapes (Game-AI) but adds little when the LLM quickly finds known heuristics (TSP). The shared board enables knowledge transfer but can hurt when early failures dominate the context; selective summarization could help.

Relation to MCTS.. Equation 1 resembles UCB in MCTS [3, 12], but our setting differs: each "move" is an LLM-generated code modification—high-cost, non-deterministic, and open-ended—versus MCTS's well-defined action spaces. Backtracking is coarser-grained, restoring full program states.

Limitations.. We use a single LLM (Kimi K2.5); results may differ with other models. Sample sizes (n=3–7) limit statistical power for high-variance tasks. The framework treats evaluation as a scalar black box, missing richer feedback (test breakdowns, execution profiles). The node selection formula uses fixed coefficients; adaptive tuning may improve performance.

6 Conclusion

SwarmOpt combines multi-agent diversity, shared memory, and tree-based backtracking for LLM-driven code optimization. Across four benchmarks, it achieves up to $+282\%$ improvement, with ablations confirming that each component contributes meaningfully on complex tasks. The framework is task-agnostic—any program with a numeric evaluation function can be optimized—and supports both sequential and parallel execution. Code and benchmarks are publicly available.*

References

[1] Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V Le, Christopher Ré, and Azalia Mirhoseini. Large language monkeys: Scaling inference compute with repeated sampling. *arXiv preprint arXiv:2407.21787*, 2024.

[2] Angelica Chen, David Dohan, and David So. Evoprompting: Language models for code-level neural architecture search. *Advances in Neural Information Processing Systems*, 36, 2023.

[3] Rémi Coulom. Efficient selectivity and backup operators in Monte-Carlo tree search. In *International Conference on Computers and Games*, pages 72–83, 2006.

[4] Yilun Du, Shuang Li, Antonio Torralba, Joshua B Tenenbaum, and Igor Mordatch. Improving factuality and reasoning in language models through multiagent debate. *arXiv preprint arXiv:2305.14325*, 2023.

[5] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, et al. MetaGPT: Meta programming for a multi-agent collaborative framework. *arXiv preprint arXiv:2308.00352*, 2024.

[6] Dong Huang, Jie Bu, Jie M Zhang, Michael Luck, and Heming Cui. AgentCoder: Multi-agent-based code generation with iterative testing and optimisation. *arXiv preprint arXiv:2312.13010*, 2024.

[7] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, et al. Competition-level code generation with AlphaCode. *Science*, 378(6624):1092–1097, 2022.

[8] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. *Advances in Neural Information Processing Systems*, 36, 2023.

*Repository URL withheld for anonymous review.

[9] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, et al. Chatdev: Communicative agents for software development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024.

[10] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M Pawan Kumar, Emilien Dupont, Francisco JR Ruiz, Jordan S Ellenberg, Pengming Wang, Omar Fawzi, et al. Mathematical discoveries from program search with large language models. In *Nature*, volume 625, pages 468–475, 2024.

[11] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36, 2023.

[12] David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, et al. Mastering the game of Go without human knowledge. *Nature*, 550(7676):354–359, 2017.

[13] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. *Advances in Neural Information Processing Systems*, 36, 2023.

A Per-Run Results

We report individual run scores for all configurations to enable full reproducibility assessment.

A.1 Game-AI (n=7)

Table 2: Game-AI per-run scores (win rate %, maximize). Baseline: 25.6%.

Config	R1	R2	R3	R4	R5	R6	R7	Mean	Std
Full	62.5	75.0	75.0	100.0	100.0	100.0	100.0	87.5	16.1
No-Backtrack	73.8	73.8	73.8	75.0	83.1	85.6	98.8	80.5	9.4
Single-Agent	71.3	71.9	73.8	75.0	86.3	87.5	97.5	80.4	10.1

A.2 Bio-Opt (n=3)

Table 3: Bio-Opt per-run scores (motif F1, maximize). Baseline: 39.6.

Config	R1	R2	R3	Mean	Std
Full	42.8	54.0	52.9	49.9	6.2
No-Backtrack	39.9	44.4	51.7	45.3	6.0
Single-Agent	39.6	44.6	45.6	43.2	3.2

A.3 Scheduler (n=7)

Table 4: Scheduler per-run scores (weighted tardiness, minimize). Baseline: 213,837.

Config	R1	R2	R3	R4	R5	R6	R7	Mean	Std
Full	133K	68K	92K	82K	79K	105K	55K	88K	26K
No-Backtrack	46K	88K	79K	97K	94K	125K	66K	85K	25K
Single-Agent	72K	93K	87K	105K	82K	106K	157K	100K	28K

A.4 TSP (n=3)

A.5 Round Scaling — Game-AI (n=3)

A.6 Backtrack Depth — Game-AI (n=3)

A.7 Board Ablation (n=3)

A.8 Parallel vs. Sequential — Game-AI (n=3)

Table 5: TSP per-run scores (tour distance, minimize). Baseline: 592.7.

Config	R1	R2	R3	Mean	Std
Full	493.4	497.7	493.4	494.8	2.5
No-Backtrack	493.4	502.4	496.9	497.6	4.5
Single-Agent	496.1	495.4	495.4	495.6	0.5

Table 6: Per-run scores for round scaling on Game-AI (full config).

Rounds	R1	R2	R3	Mean	Std
2	71.3	58.8	73.1	67.7	7.8
4	86.3	100.0	70.0	85.4	15.0
8	87.5	100.0	86.3	91.3	7.6
12	100.0	93.8	100.0	97.9	3.6

Table 7: Per-run scores for backtrack threshold k on Game-AI (full config, 6 rounds).

k	R1	R2	R3	Mean	Std	Backtracks
1	87.5	68.8	80.6	79.0	9.5	2, 3, 3
2	87.5	100.0	86.3	91.3	7.6	0, 1, 0
5	76.3	87.5	73.8	79.2	7.3	0, 0, 0

Table 8: Board ablation: full system vs. no-board (board summary disabled in prompts).

Task	Config	R1	R2	R3	Mean	Std
Game-AI	Full	75.0	100.0	100.0	91.7	14.4
	No-Board	86.3	100.0	84.4	90.2	8.5
Bio-Opt	Full	42.8	54.0	52.9	49.9	6.2
	No-Board	64.3	61.9	41.8	56.0	12.4
TSP	Full	493.4	497.7	493.4	494.8	2.5
	No-Board	493.4	493.4	497.7	494.8	2.5

Table 9: Parallel vs. sequential execution on Game-AI (full config, 6 rounds).

Mode	R1	R2	R3	Mean	Std	Time (s)
Sequential	86.3	85.0	73.8	81.7	6.9	1122
Parallel	87.5	75.0	81.9	81.5	6.3	429